

CE 310

Python Tutorial — Multiple Linear Regression

Week 10 · Session 2 · Hands-on Walkthrough

University of Arizona · Dr. Wooyoung Jung · Fall 2026

Today's Plan — Four New Skills, Then the In-Class Exercise

By the end of this walkthrough you will know how to:

1. Use `statsmodels` to fit regression models with a formula string
2. Read the **summary table** — extract R^2 , Adj. R^2 , coefficients, and p-values
3. Add predictors and compare models (BH vs. Season vs. interaction)
4. Fit an **interaction model** and extract separate slopes for occupied and unoccupied hours

Assumed from Week 9: you can load the CSV, call `corr()` and `linregress()`, and plot a scatter with a regression line.

File needed: `Week5_D0E_MedOffice_ZoneTemp_2023.csv` — same file as Week 9.

Step 1 — Setup: One New Library

📌 **New this week: statsmodels** — produces full regression tables with p-values and confidence intervals. It is pre-installed in Google Colab.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import statsmodels.formula.api as smf    # ← new

df = pd.read_csv('Week5_D0E_MedOffice_ZoneTemp_2023.csv')

# Create the occupancy dummy variable: 1 = occupied, 0 = unoccupied
df['BH'] = (df['BusinessHours'] == 'Yes').astype(int)

print(f"Rows: {df.shape[0]}")
print(f"Occupied hours (BH=1): {df['BH'].sum()}")
```

Expected: Rows: 8760 · Occupied hours: 2600

Running locally instead of Colab? `pip install statsmodels`

Step 2 — The statsmodels Formula Interface

`statsmodels` uses a **formula string** instead of separate x and y arrays. The formula describes the model in plain English:

```
model2 = smf.ols('Cooling_kWh ~ Outdoor_Temp_F + BH', data=df).fit()  
print(model2.summary())
```

Reading the formula string:

- `Cooling_kWh` — the outcome variable (left of `~`)
- `Outdoor_Temp_F + BH` — the predictors (right of `~`)
- `smf.ols(...)` — ordinary least squares regression
- `.fit()` — runs the fitting calculation

Compare to Week 9: `stats.linregress(x, y)` accepted only one predictor and returned five numbers. `statsmodels` accepts any number of predictors and returns a full summary table.

Step 3 — What the Summary Table Shows

Running `print(model2.summary())` produces a multi-section output. Focus on these parts:

Top section (first box):

- `R-squared` → 0.697
- `Adj. R-squared` → 0.697
- `No. Observations` → 8760

Coefficient table (middle box):

	coef	P> t	[0.025, 0.975]
Intercept	-55.5	0.000	...
Outdoor_Temp_F	1.026	0.000	...
BH	17.9	0.000	...

All three p-values say 0.000 — this means $p < 0.0005$, which statsmodels rounds to 0.000. Every predictor and the intercept are statistically significant.

Step 4 — Extract Numbers Programmatically

You do not need to read the summary table by eye for every calculation. Access individual values directly:

```
# R² and Adjusted R²
r2      = round(model2.rsquared, 4)
adj_r2  = round(model2.rsquared_adj, 4)
print(f"R²      = {r2}")
print(f"Adj. R² = {adj_r2}")

# Individual coefficients
bh_coef  = round(model2.params['BH'], 3)
temp_slope = round(model2.params['Outdoor_Temp_F'], 4)
intercept = round(model2.params['Intercept'], 2)
print(f"BH coefficient:    {bh_coef} kWh/hr")
print(f"Temp slope:       {temp_slope} kWh/°F")
print(f"Intercept:         {intercept} kWh")
```

Expected: $R^2 = 0.697$ · BH = 17.9 kWh/hr · Temp slope = 1.026 kWh/°F

`model2.params` is a dictionary. Key = predictor name as written in the formula string.

Step 5 — Collinearity: Season vs. BusinessHours

 Fit Model 3 (+Season) and compare its R^2 to Model 2 (+BH).

```
# C(Season) tells statsmodels to treat Season as a category
model3 = smf.ols('Cooling_kWh ~ Outdoor_Temp_F + C(Season)', data=df).fit()
r2_m3 = round(model3.rsquared, 4)

print(f"Model 2 R2 (+BH):      {round(model2.rsquared, 4)}")
print(f"Model 3 R2 (+Season): {r2_m3}")
print(f"Week 9 baseline:      0.6010")
```

Expected: Model 2 = 0.697 · Model 3 = 0.611

Season adds only 1 percentage point on top of outdoor temperature. BusinessHours adds 9.6 percentage points. Both try to explain what outdoor temperature cannot — but Season and outdoor temperature are nearly synonymous: summer = hot, winter = cold. **BusinessHours** carries **genuinely new information** that temperature does not.

Step 6 — Fit the Interaction Model

```
# Outdoor_Temp_F * BH creates: Outdoor_Temp_F, BH, AND their product
model5 = smf.ols('Cooling_kWh ~ Outdoor_Temp_F * BH', data=df).fit()
print(model5.summary())
```

The coefficient table now has **four rows**:

Row	What it means
Intercept	Predicted cooling at 0°F, unoccupied
Outdoor_Temp_F	Temperature slope when unoccupied (BH = 0)
BH	Shift in intercept when occupied vs. unoccupied
Outdoor_Temp_F:BH	Extra slope per degree when occupied

Full slope when occupied = Outdoor_Temp_F coef + Outdoor_Temp_F:BH coef

Step 7 — Extract the Two Slopes

```
slope_unoccupied = round(model5.params['Outdoor_Temp_F'], 4)
extra_slope      = round(model5.params['Outdoor_Temp_F:BH'], 4)
slope_occupied   = round(slope_unoccupied + extra_slope, 4)

print(f"Unoccupied slope:           {slope_unoccupied} kWh/°F")
print(f"Extra slope when occupied: {extra_slope} kWh/°F")
print(f"Occupied slope:             {slope_occupied} kWh/°F")
print(f"Interaction model R²:       {round(model5.rsquared, 4)}")
```

Expected: unoccupied ≈ 0.71 · occupied ≈ 1.78 · $R^2 \approx 0.795$

The occupied building is 2.5× more sensitive to outdoor temperature. Each extra degree of heat drives much more cooling when people, computers, and lights are running than when the building is empty.

Step 8 — Residual Plot Comparison

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

for ax, model, title in zip(
    axes, [model2, model5],
    ['Model 2: + BusinessHours', 'Model 5: Interaction']):
    resid = model.resid
    ax.scatter(df['Outdoor_Temp_F'], resid,
               alpha=0.1, s=4, color='orange')
    ax.axhline(0, color='black', linewidth=1, linestyle='--')
    ax.set_xlabel('Outdoor Temperature (°F)')
    ax.set_ylabel('Residual (kWh/hr)')
    ax.set_title(f'{title}\nR2={model.rsquared:.4f}')

plt.tight_layout()
plt.show()
```

What to look for: Model 2 still shows two bands of residuals (occupied above, unoccupied below the zero line). Model 5 brings these together — the fan shape and the two-band structure are mostly gone.

Step 9 — AIC Comparison

```
# AIC: lower = better fit per complexity
model1 = smf.ols('Cooling_kWh ~ Outdoor_Temp_F', data=df).fit()
model4 = smf.ols('Cooling_kWh ~ Outdoor_Temp_F + BH + C(Season)',
                 data=df).fit()

models = [('Simple (Wk9)', model1),
          ('+ BH (M2)', model2),
          ('+ Season (M3)', model3),
          ('+ BH+Season (M4)', model4),
          ('Interaction (M5)', model5)]

for name, m in models:
    print(f"{name:22s} R²={m.rsquared:.4f} AIC={m.aic:,.0f}")
```

Model 5 will have the lowest AIC by a large margin. Model 3 (+Season) will have a higher AIC than the simple model — adding a collinear predictor costs complexity without sufficient payoff.

Quick Check — Before You Start the In-Class Exercise

Verify your numbers, then open `Wk10.03_In_Class_Exercise.ipynb`

Confirm all four values:

- **R^2 (Model 2, +BH)** = 0.697 (up from 0.601 in Week 9)
- **BH coefficient** ≈ 17.9 kWh/hr ($p \approx 0.000$)
- **Unoccupied slope** ≈ 0.71 kWh/°F · **Occupied slope** ≈ 1.78 kWh/°F
- **R^2 (interaction model)** ≈ 0.795

Question: The raw difference in mean cooling between occupied and unoccupied hours is about 28 kWh/hr (from the group means). The regression BH coefficient is only 17.9 kWh/hr. Why are these two numbers different, and which one is a better estimate of the true occupancy effect?

Quick Reference — Week 10 New Python Tools

Task	Code
Import statsmodels	<code>import statsmodels.formula.api as smf</code>
Print full summary	<code>model.summary()</code>
Get R ²	<code>model.rsquared</code>
Get AIC	<code>model.aic</code>
Count distinct values	<code>df['col'].nunique()</code>

Task	Code
Get a coefficient	<code>model.params['x1']</code>
Get a p-value	<code>model.pvalues['x1']</code>
Get residuals	<code>model.resid</code>
Interaction term	<code>x1 * x2</code> in formula
Coefficient 95% CI	<code>model.conf_int().loc['x1']</code>

Before You Submit

⚠ Following this submission format exactly is essential — with a class this size, grading depends on it. Submissions that don't comply will be penalized.

1. **Run all cells first.** In Colab: Runtime → Run all. Wait until every cell shows a checkmark — no ⏳ spinners remaining.
2. **Download as .ipynb.** File → Download → Download .ipynb. Do not submit PDF or .py.
3. **Do not edit the `print` lines.** Lines like `print(f"ANSWER_A1 = {ANSWER_A1}")` must remain exactly as written — changing or deleting them means that part of your answer will not be counted.
4. **File name —** `CE310_W10_<NetID>.ipynb`, e.g., `CE310_W10_abc123.ipynb`.
5. **Upload** the `.ipynb` file to D2L Dropbox by **9:00 AM next Wednesday**.